Chapter 2

Algorithms

Contents

1. Introduction to algorithm
2. Flowchart
3. Pseudocode
4. LARP
5. Variables
6. Data Types
7. Review questions

1. Introduction to Algorithm

- In programming, what is an algorithm?
  - *is a method to solve a problem that consists of exactly defined instructions*
  - *is a step-by-step procedure for computations*
- A procedure for solving a problem specifies:
  - the actions to be executed, and
  - the order in which these actions are to be executed

1. Introduction to Algorithm

- An algorithm to add two numbers entered by user:

step 1: start
step 2: get user input for number 1
step 3: get user input for number 2
step 4: add number 1 and number 2
step 5: display the result
step 6: stop

1. Introduction to Algorithm

- An algorithm to find your age by input born year:

step 1: start
step 2: get input born year
step 3: minus current year with born year
step 4: display result
step 5: stop

1. Introduction to Algorithm

- An algorithm to find the price after discount of some pens you buy (supposed all pens is the same):

  step 1: start

  step 2: get quantity of product you bought

  step 3: get unit price for the pen

  step 4: get discount price for all pens

  step 5: calculate total price by time the quantity and unit price

  step 6: calculate price after discount by discount price from total price

  step 7: display result

  step 8: stop

1. Introduction to Algorithm

- A good algorithm should have the following qualities:
  - *Inputs and outputs should be defined precisely*
  - *Each steps in algorithm should be clear and unambiguous*
  - *Algorithm should be most effective among many different ways to solve a problem*
  - *An algorithm shouldn't have computer code*
  - *An algorithm should be written in such a way that, it can be used in similar programming languages*

1. Introduction to Algorithm

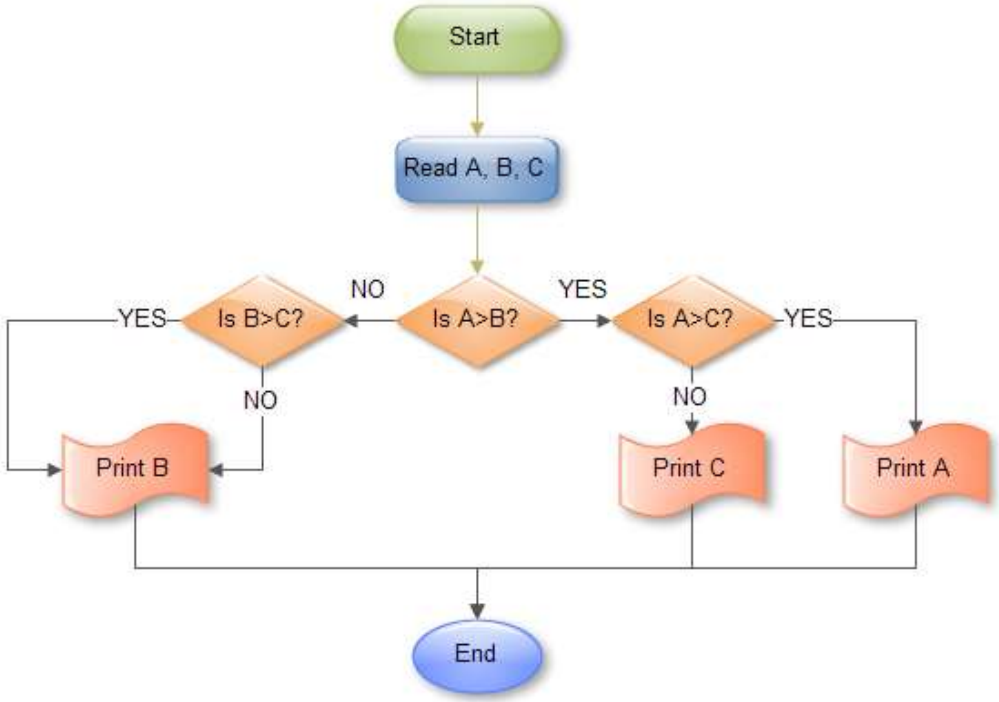
- Performing an algorithm might or might not solve a specified problem if the algorithm is flawed or not appropriate to the problem.
- There are two main, closely related tasks in study of algorithms:
  - *designing an algorithm to solve a specific problem*
  - *analyzing a given algorithm*
- Algorithms can be expressed by task list, pseudo-code and flowchart.

2. Flowchart

- A flowchart is a type of diagram that presents an algorithm or process.
- A flowchart is a graphic tool for providing a solution to a problem.
- Flowcharts were introduced by Frank Gilberth in 1921
  - The first name was called “Process Flow Charts”
- A flowchart shows the steps as boxes of various kinds, and their order by connecting them with arrows.

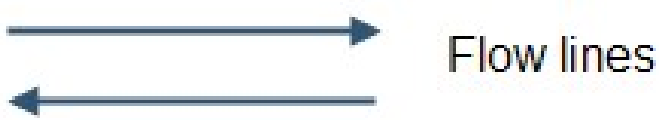
2. Flowchart

- This is an example of a flowchart.



2. Flowchart

- Flowchart symbols
  - Flow lines
    - Used to connect flowchart symbols together.



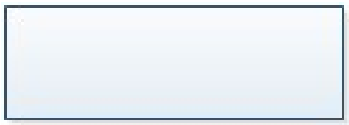
2. Flowchart

- Flowchart symbols
  - Terminator
    - Indicates START, STOP, END or RETURN
    - Allows flowchart users know where and when to start and stop

Terminator

2. Flowchart

- Flowchart symbols
  - Process
    - Used to describe a computer processing step
    - Typically contains an equation or formula
    - The word if, and, or will never be used inside this symbol



Process

2. Flowchart

- Flowchart symbols
  - Decision
    - Used to make decision
    - Sentences inside this symbol will end with question mark

Decision

2. Flowchart

- Flowchart symbols
  - Data (input / output)
    - Contains words to describe data or information
    - Whether it is input or output

Data (input / output)

2. Flowchart

- Flowchart symbols
  - Document
    - Represents document
    - Represents report

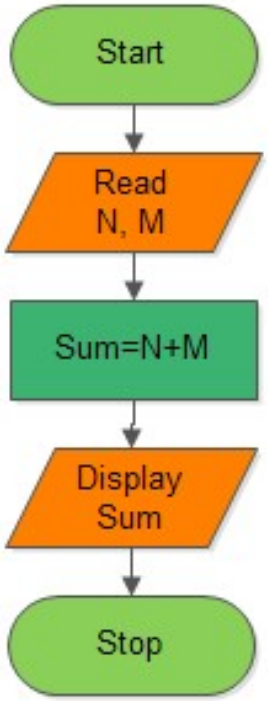
Document

2. Flowchart

- Flowchart symbols
  - Display
    - Machine display

Display

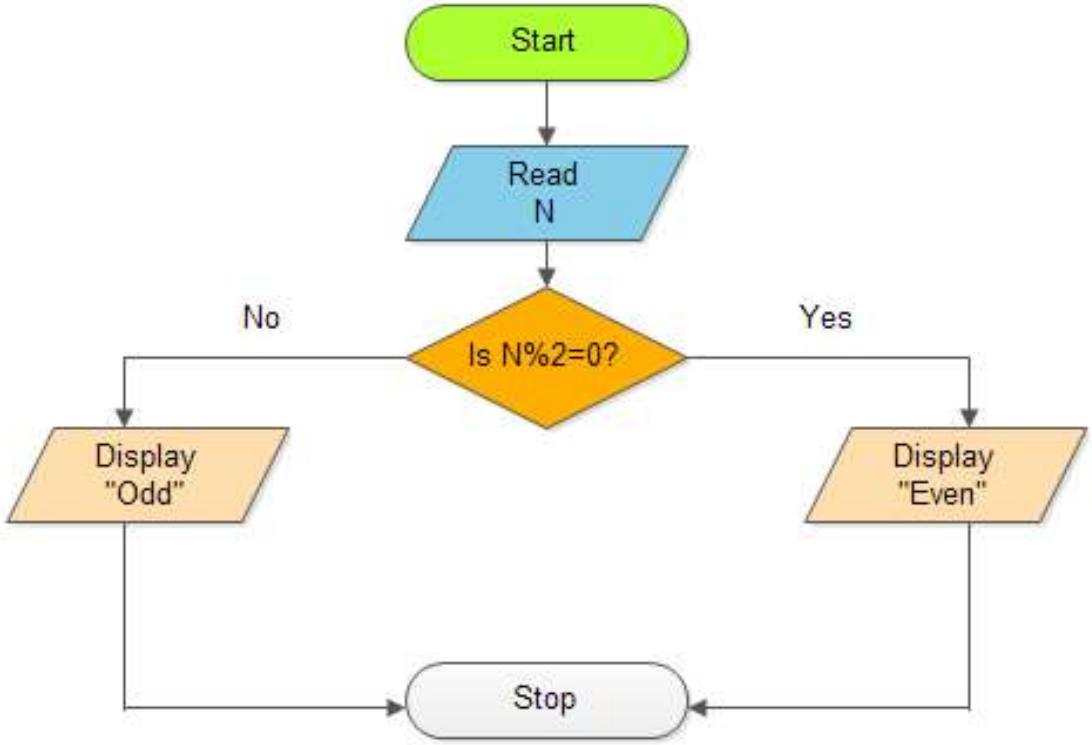
2. Flowchart



A flowchart to find the sum of two integers by user input

2. Flowchart

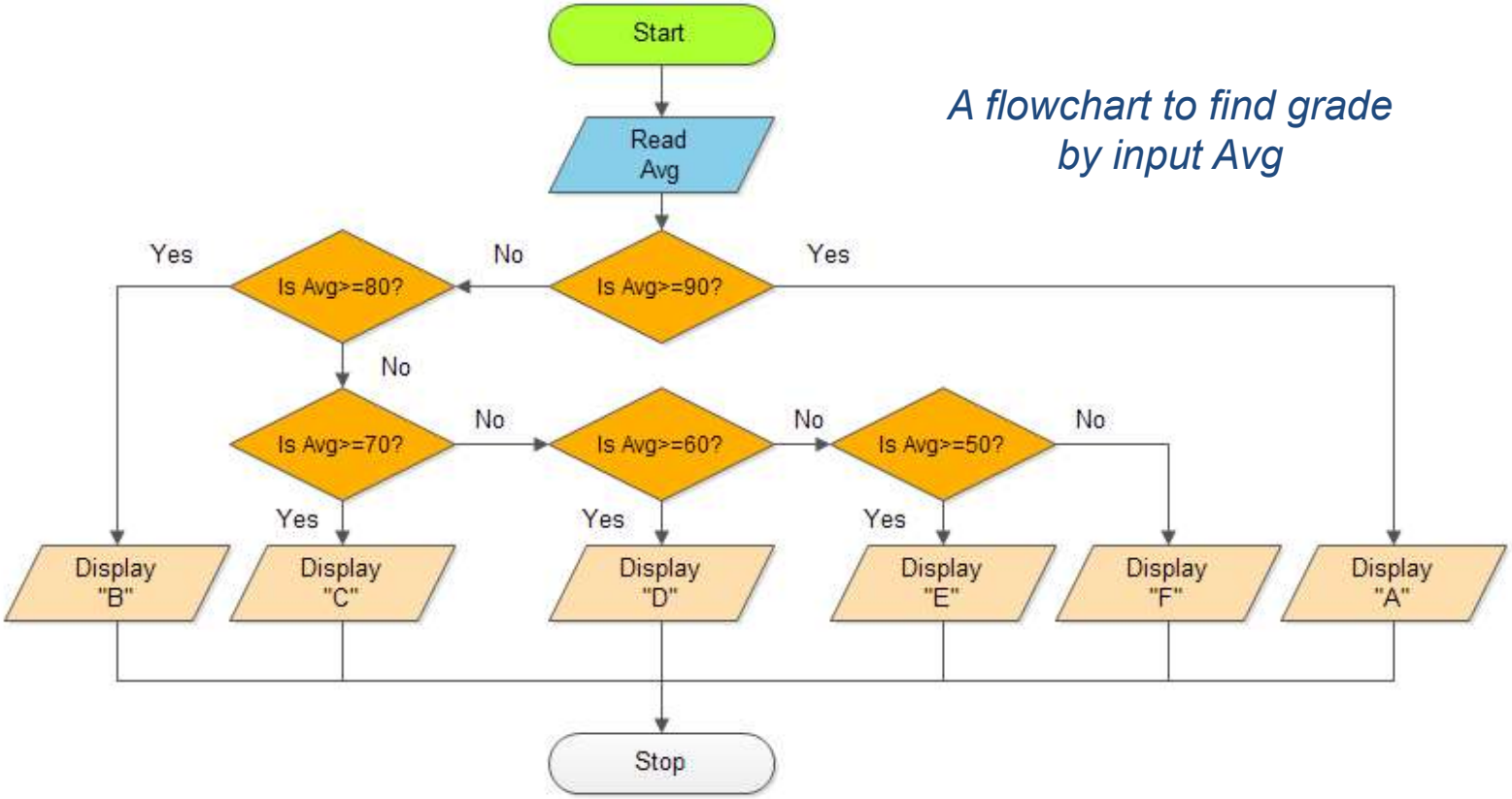
- A flowchart to define a number input odd or even:



2. Flowchart

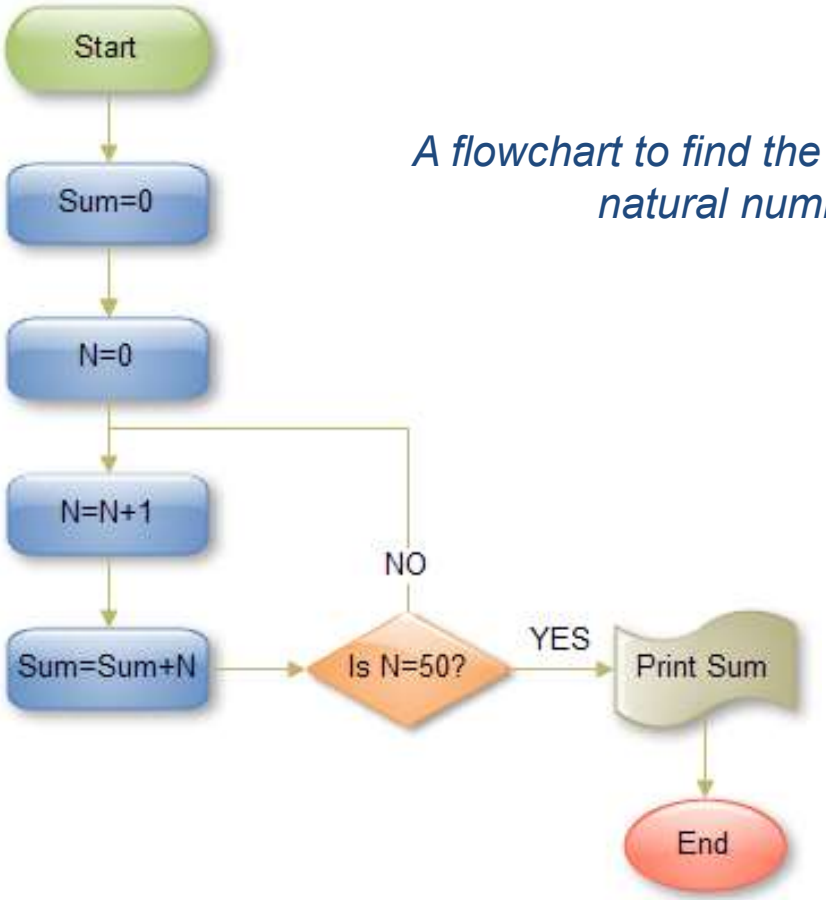
- Flowchart rules:
  - must start somewhere
  - must end somewhere
  - must unfailingly proceed from start to end
  - flow lines never cross each other
  - flowchart symbols can only have one entry point

2. Flowchart



A flowchart to find grade by input Avg

2. Flowchart



A flowchart to find the sum of first 50 natural numbers.

3. Pseudo-code

- Pseudocode is an artificial and informal language that helps programmers develop algorithms.
- Pseudocode is a "text-based" detail (algorithmic) design tool.
- Pseudocode was written in a form that can easily be converted into real programming statements.
- Pseudocode can not be compiled nor executed, and there are no real formatting or syntax rules.

3. Pseudo-code

- Let consider this simple example,

If student's grade is greater than or equal to 60 then

    Print "Passed"

Else

    Print "Failed"

Endif

3. Pseudo-code

- Some rules of pseudocode:
  - *we still use assignment statement in pseudocode*
  - *pseudocode commands are capitalized such as:*
    - *Get, Set, Input, Display, Print, If….*
  - *variable names will not have blank spaces*
  - *use underscore if variables have more than one word*
  - *the first word of the variable name is capitalized*
  - *control is transferred to sub-module when the sub-module name is encountered*
  - *pseudocode main programs end with the statement End*
  - *sub-modules end with the statement Return*

3. Pseudo-code

- Area of a Circle Program

Get Radius
Set PI=3.14
*Area = Radius * Radius * PI*
Display Radius
Display Area
End

3. Pseudo-code

- Cylinder Volume Calculation

```
Set PI=3.14159
Input Radius
Input Height
Volume=PI * Radius * Radius * Height
Print "For cylinder height of:" Height
Print "For cylinder radius of:" Radius
Print "The volume is: " Volume
End
```

3. Pseudo-code

- Email Login Program

```
Input Email_Address
Input Password
if Email_Addrss is existed then
        if Password is correct then
                Goto Mail_Box
        Else
                Print "Invalid Password!"
        Endif
Else
        Pint "Invalid Email Address!"
Endif
End
```

3. Pseudo-code

- Grade Evaluation Program

```
Input Grade
If Grade is greater than or equal to 90 then
        Print "A"
Else
        If Grade is greater than or equal to 80 then
                Print "B"
        Else
                If Grade is greater than or equal to 70 then
                        Print "C"
                Else
                        If Grade is greater than or equal to 60 then
                                Print "D"
                        Else
                                If Grade is greater than or equal to 50 then
                                        Print "E"
                                Else
\                                       Print "F"
                                Endif
                        Endif
                Endif
        Endif
Endif
End
```

3. Pseudo-code

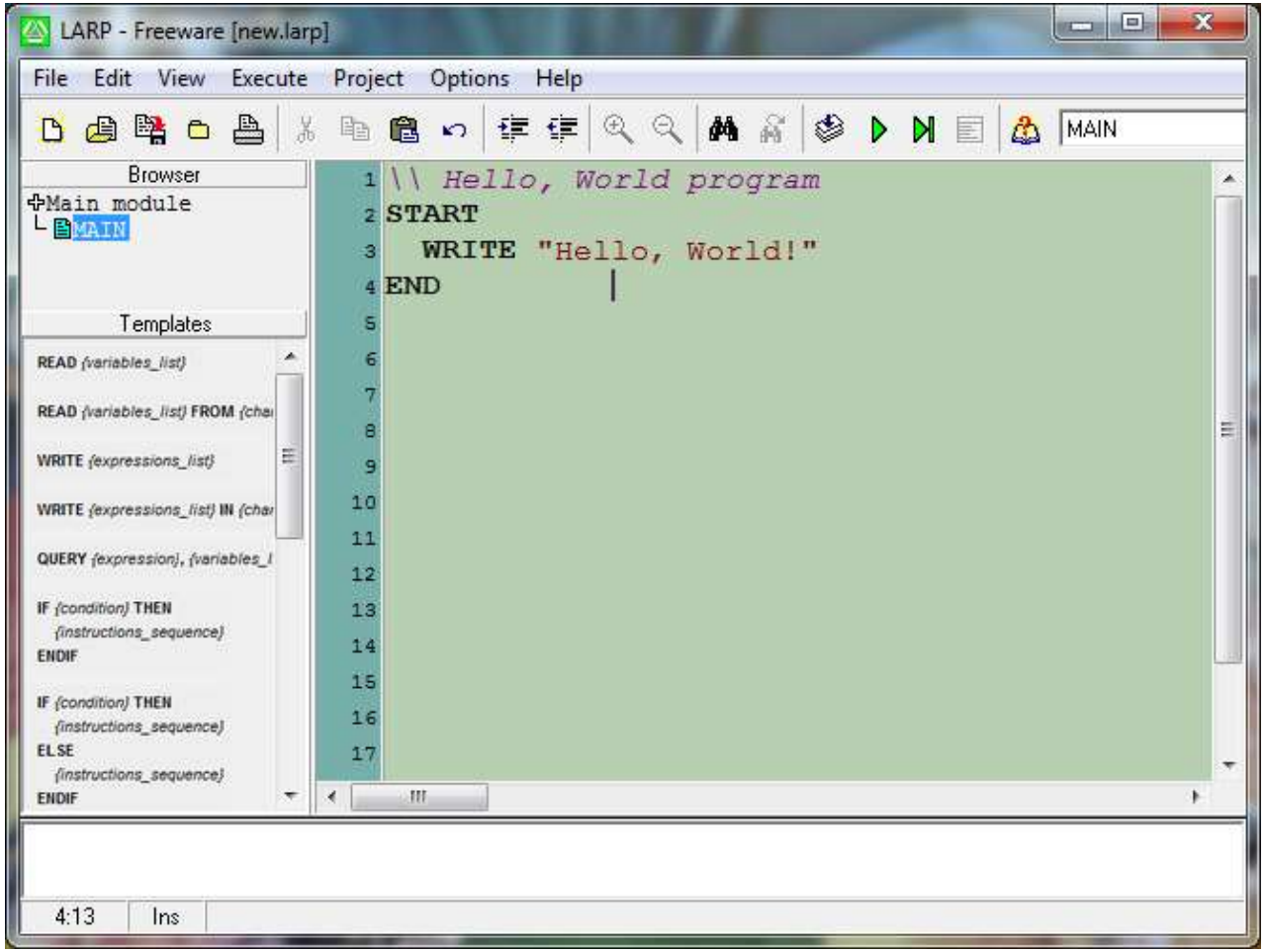
- Even Finder From 1 To 100 Program

```
Set Counter  to 1
While Counter less than or equal to 100 do
        If Counter Mod 2 equal to 0 then
                        Print Counter
        Endif
        Add Counter by 1
Endwhile
End
```

4. LARP

- What is LARP?
  - stands for *Logics of Algorithms and Resolution of Problems*
  - is used to type your algorithms both flowchart and pseudo-code
  - is semi-natural syntax of English language
- The following are some features of LARP:
  - Sequential structure
  - Variables
  - Containers
  - Decision making
  - Iteration
  - Modules

4. LARP


LARP - Freeware [new.larp]
File Edit View Execute Project Options Help
MAIN
Browser
Main module
MAIN
Templates
READ {variables_list}
READ {variables_list} FROM {char
WRITE {expressions_list}
WRITE {expressions_list} IN {char
QUERY {expression}, {variables_l
IF {condition} THEN
{instructions_sequence}
ENDIF
IF {condition} THEN
{instructions_sequence}
ELSE
{instructions_sequence}
ENDIF
\\ Hello, World program
START
WRITE "Hello, World!"
END
4:13
Ins

5. Variables

- A variable is the memory location to store data that can be used by the program.
- A variable characteristics:
  - Data type
  - Name
  - Value
  - Size (based on data type)
  - Address (assigned by OS)
  - Operations that can be performed

5. Variables

- An identifier must not a keyword.
- It cannot contain special character like + , %...
- It cannot start by a number.
- It can not contain space.
- Its name should be meaning full.

5. Variables

```
2 START
3     math=90
4     english=80
5     BL=80
6     Total = math + English + BL
7     WRITE "Total=",Total
8 END
```

5. Variables

- Math Operators:

Addition	+	$f + 7$	`f + 7`
Subtraction	–	$p - c$	`p - c`
Multiplication	*	bm	`b * m`
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	`x / y`
Remainder	%	$r \bmod s$	`r % s`

( ) : first
*, / : second
% : third
+ , - : fourth

5. Variables

- X = -1 + 1 * 2 / (2-1) – 1
  - What X?
- X = 1 + 1/1*(2-1) – 2 – (2-1)*2-1
  - What X?
- Y = -1 – (1-1-1) – 1 * (-1 -1)
  - What Y?
- Z = 1/2/3/4 – 4/3/2/1 + (2-5*3)
  - What Z?

5. Variables

```
1  \\ Calculate total price
2  START
3      qty=0
4      up=0
5      total=0
6      WRITE "Enter Quantity:"
7      READ qty
8      WRITE "Enter Unitprice:"
9      READ up
10     total = up * qty
11     WRITE "Total = $",total
12 END
```

6. Data Types

- The data type defines the following:
  - the amount of storage allocated to variables.
  - the values that they can accept.
  - the operations that can be performed on variables.
- Basic data types:
  - Number
  - String

6. Data Types

```
\\ Main module
START
    UserName=""
    Password=""
    WRITE "Enter User Name:"
    READ UserName
    WRITE "Enter Password:"
    READ Password
    IF UserName="Admin" THEN
        IF Password="123" THEN
            WRITE "Login Success!"
        ELSE
            WRITE "Invalid Password!"
        ENDIF
    ELSE
        WRITE "Incorrect user name!"
    ENDIF
END
```

5. Review Questions

Go to exercises for chapter 2!